Mastering Data Visualization with Seaborn

This notebook provides a comprehensive guide to the Seaborn library, covering everything from distribution plots to categorical analysis and advanced faceting.

1. Setup and Sample Data

First, we import the necessary libraries and create a synthetic dataset for our examples.

In [2]:
```
# !pip install seaborn
```

In [3]:
```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
```

In [4]:
```
# Setting the theme
sns.set_theme(style="whitegrid")

# Generating a sample dataset
np.random.seed(42)
data = {
    'age': np.random.normal(35, 10, 200).astype(int),
    'salary': np.random.normal(50000, 15000, 200),
    'height': np.random.normal(170, 10, 200),
    'weight': np.random.normal(70, 15, 200),
    'gender': np.random.choice(['Male', 'Female'], 200),
    'department': np.random.choice(['Sales', 'IT', 'HR', 'Marketing'], 200),
    'experience': np.random.randint(1, 25, 200),
    'performance_score': np.random.uniform(1, 10, 200)
}
df = pd.DataFrame(data)
df.head()
```

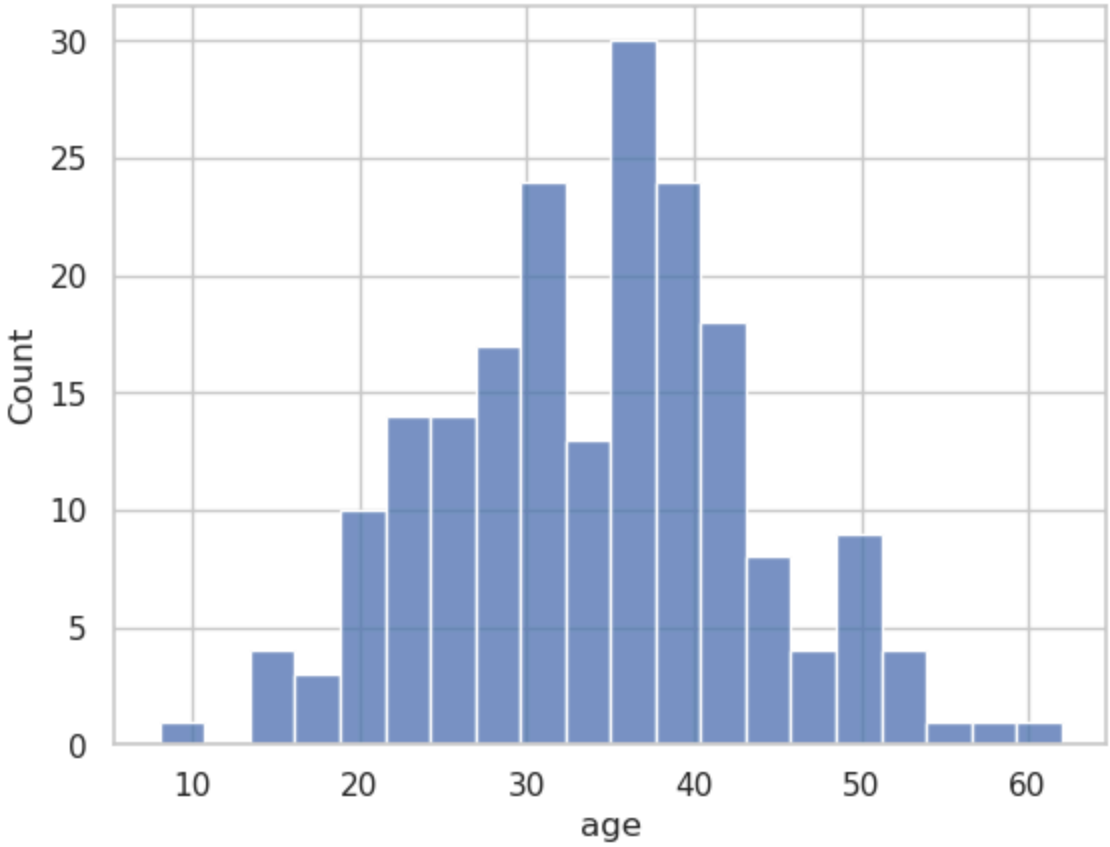
Out[4]:

	age	salary	height	weight	gender	department	experience
0	39	55366.810405	154.055723	81.354829	Female	HR	3
1	33	58411.767896	164.006250	56.167520	Male	IT	1
2	41	66245.768648	170.052437	83.044089	Female	Marketing	6
3	50	65807.030781	170.469806	90.334568	Male	HR	15
4	32	29334.959481	165.499345	76.201524	Male	Sales	2

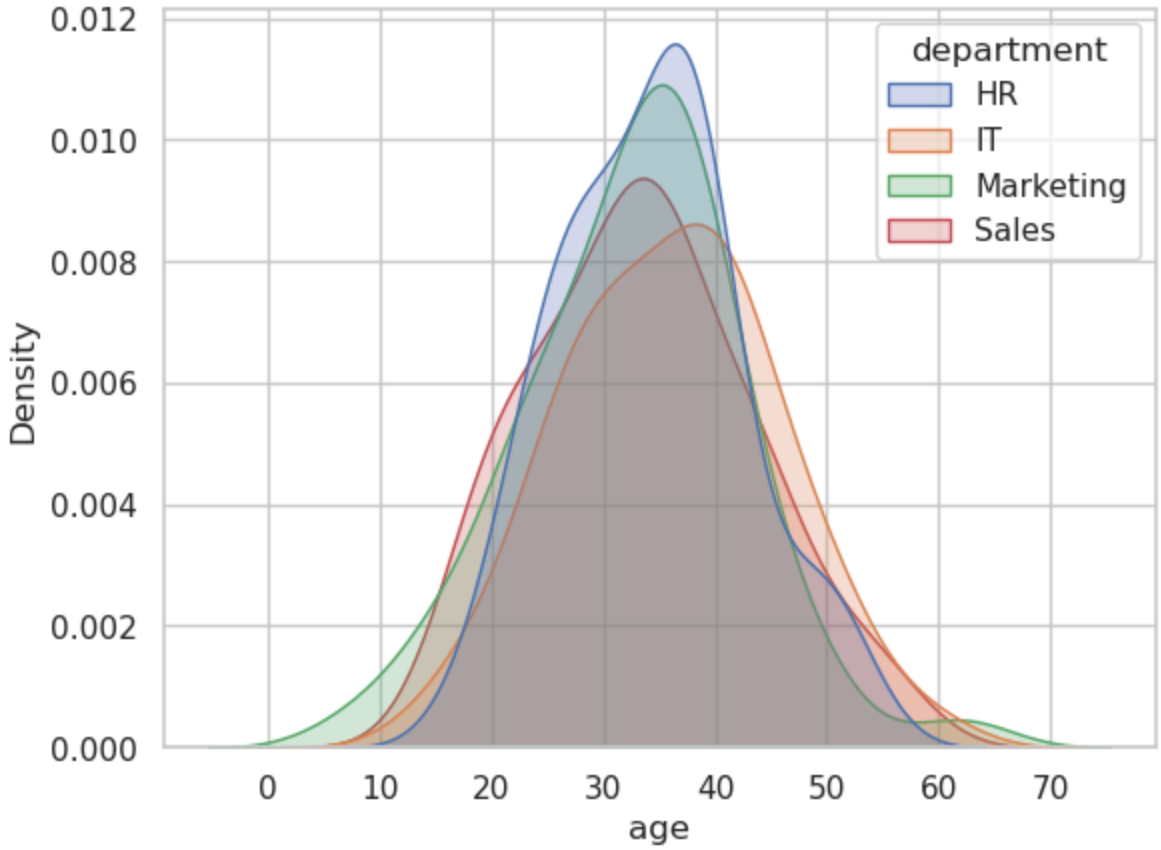
2. Distribution Plots

Used to visualize the spread and density of numeric data.

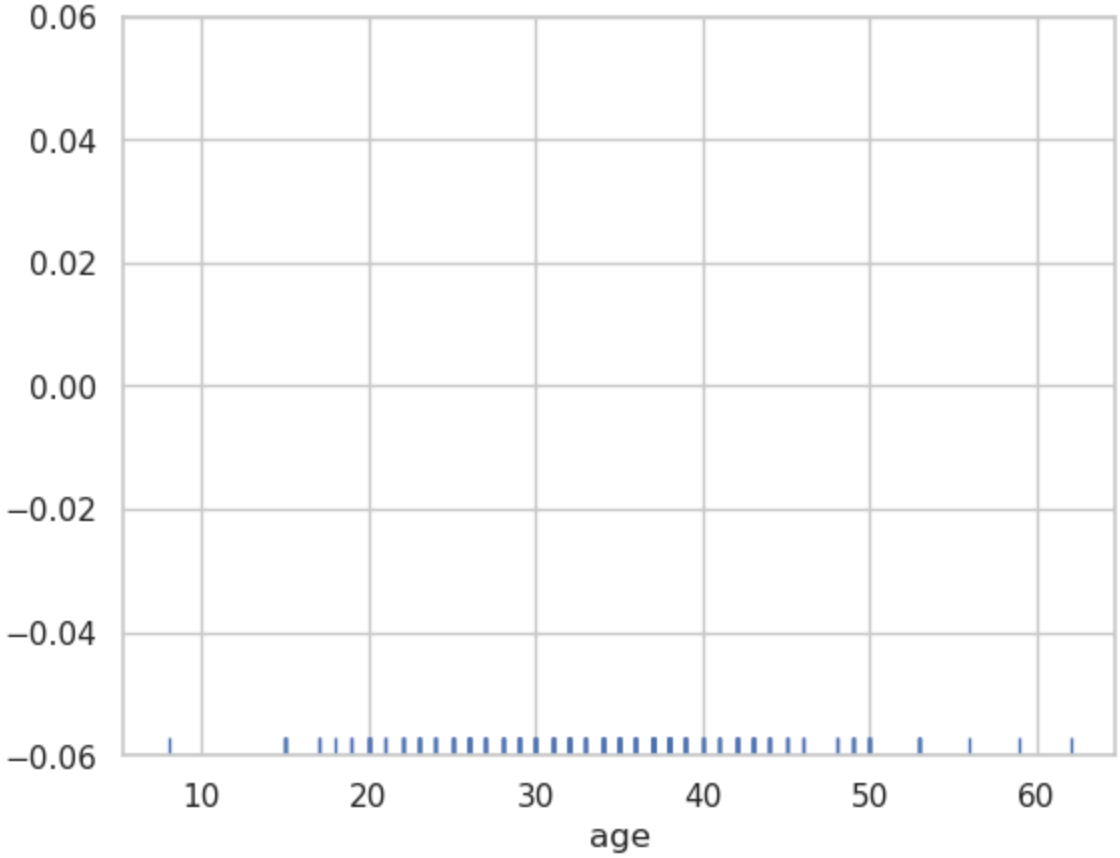
In [5]:
```
# Histogram
sns.histplot(data=df, x="age", bins=20, kde=False);
```



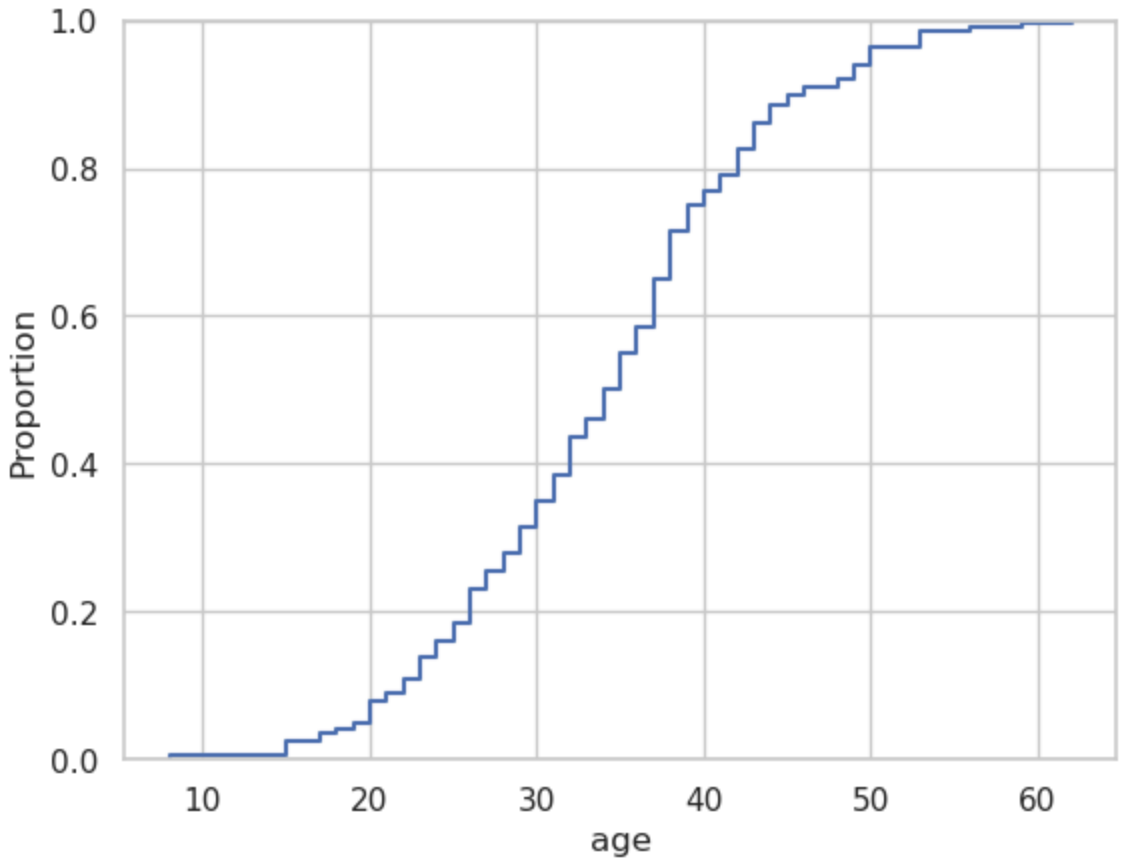
In [9]:
```
# Kernel Density Estimate (KDE)
# sns.kdeplot(data=df, x="age", fill=True);
# sns.kdeplot(data=df, x='age', fill=True, hue='gender');
sns.kdeplot(data=df, x='age', fill=True, hue='department');
```



In [10]:
```
# Rug Plot
sns.rugplot(data=df, x="age");
```



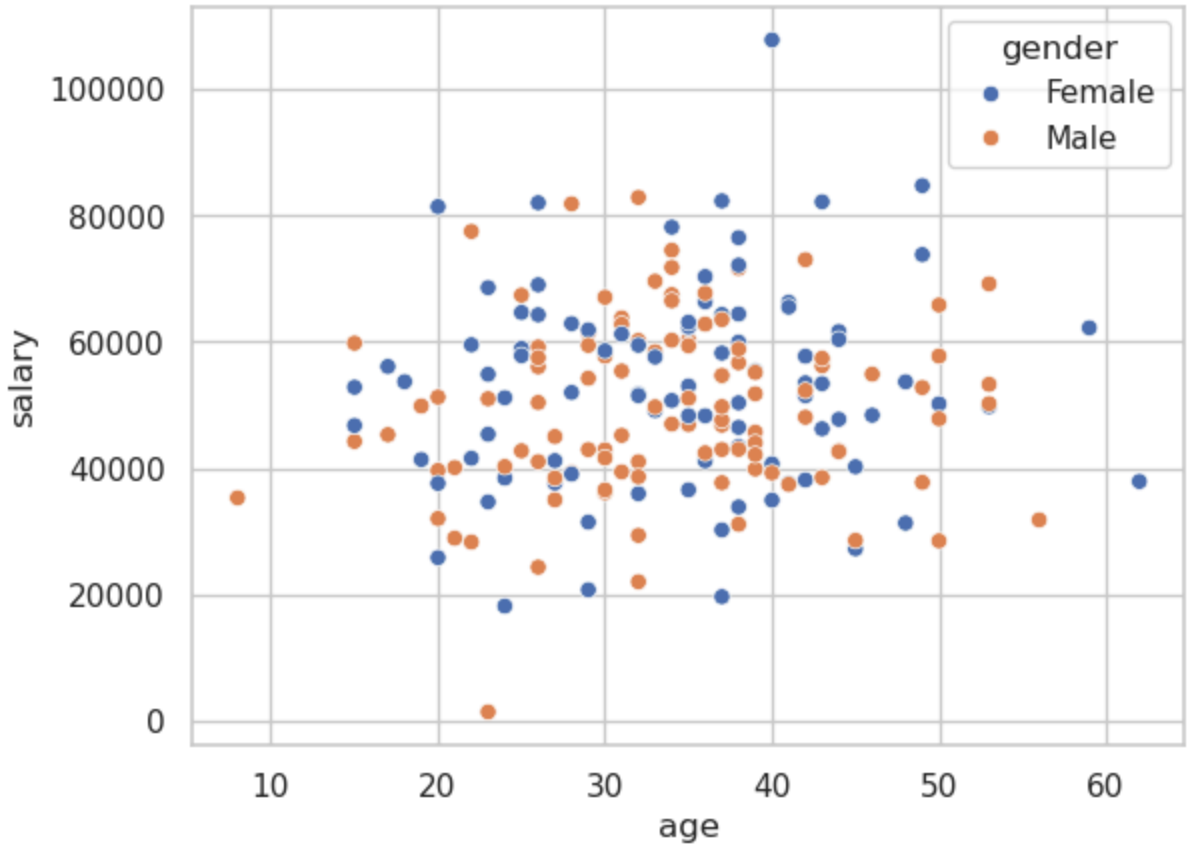
In [11]:
```
# ECDF Plot (Empirical Cumulative Distribution)
sns.ecdfplot(data=df, x="age");
```



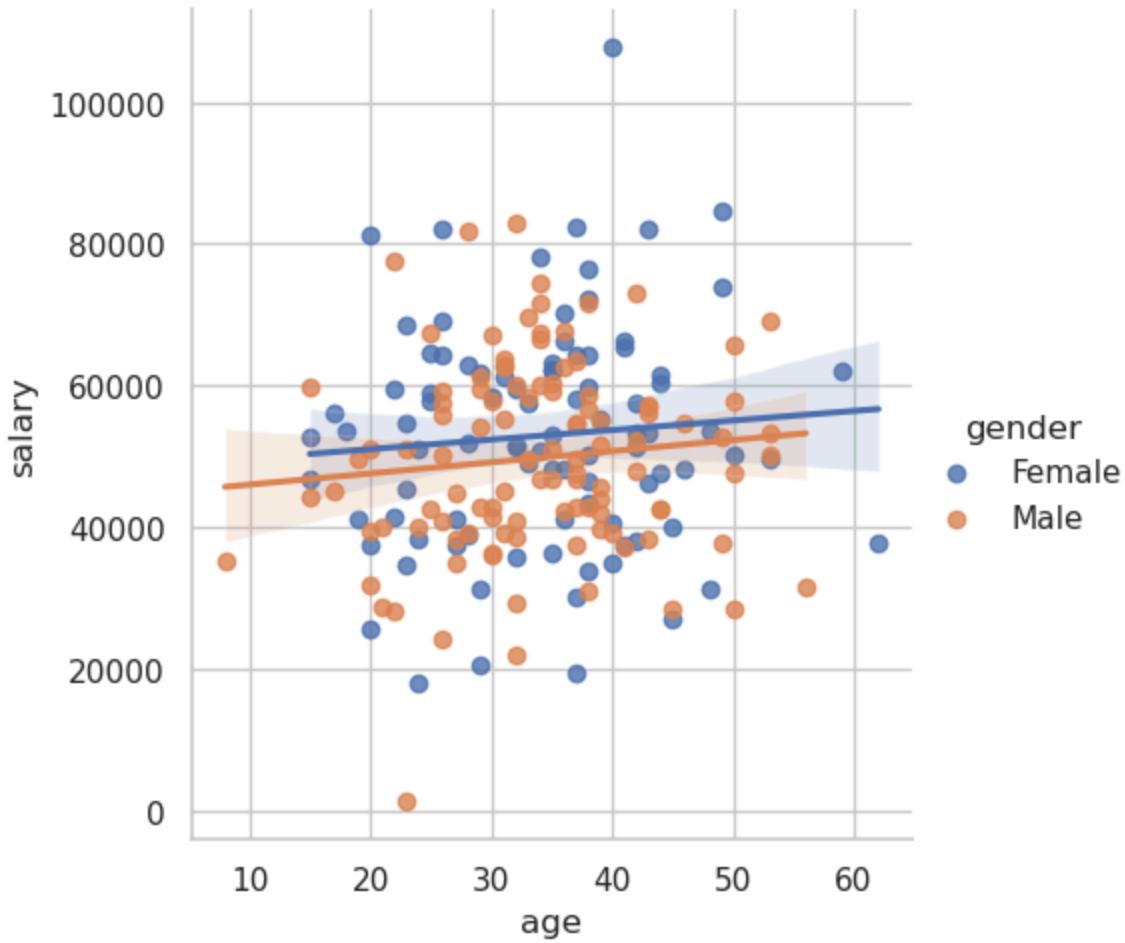
3. Relational Plots

Used to identify relationships between multiple variables.

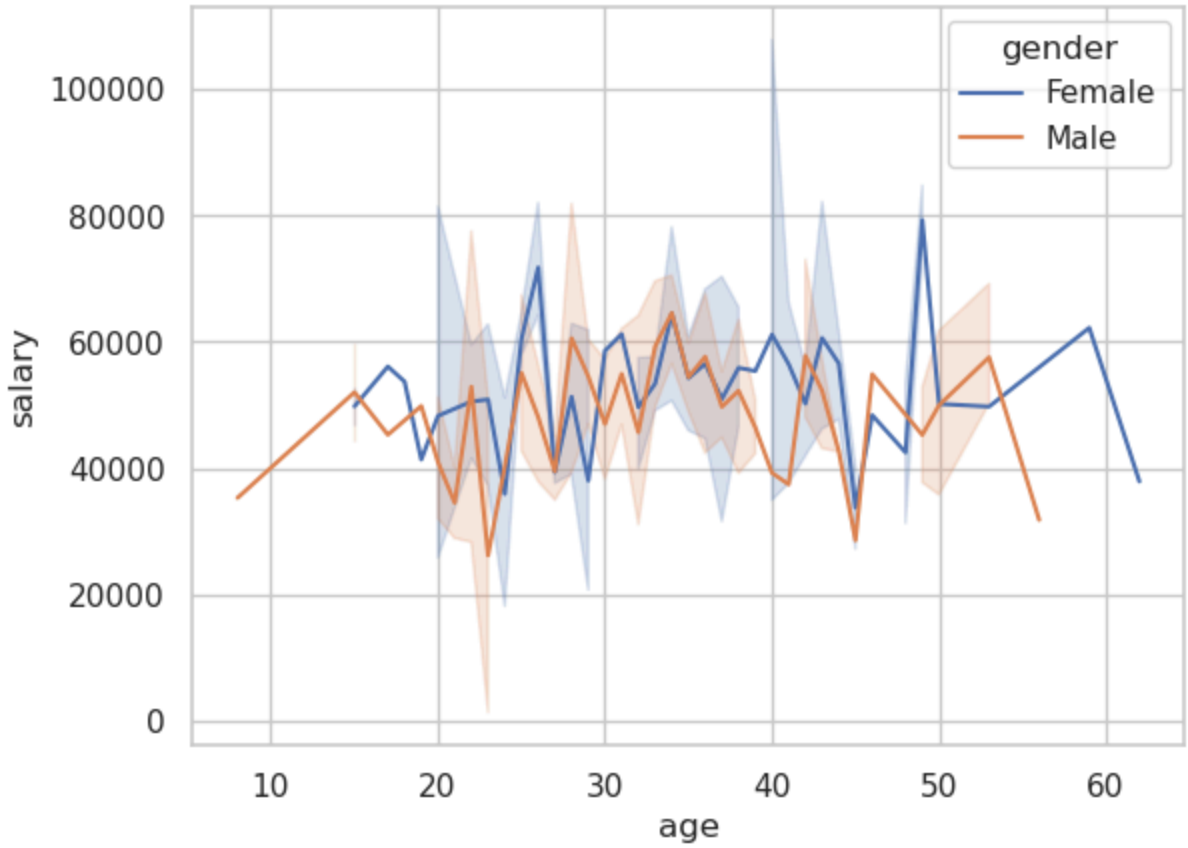
In [13]:
```
# Scatter Plot
# sns.scatterplot(data=df, x='age', y='salary');
sns.scatterplot(data=df, x='age', y='salary', hue='gender');
```



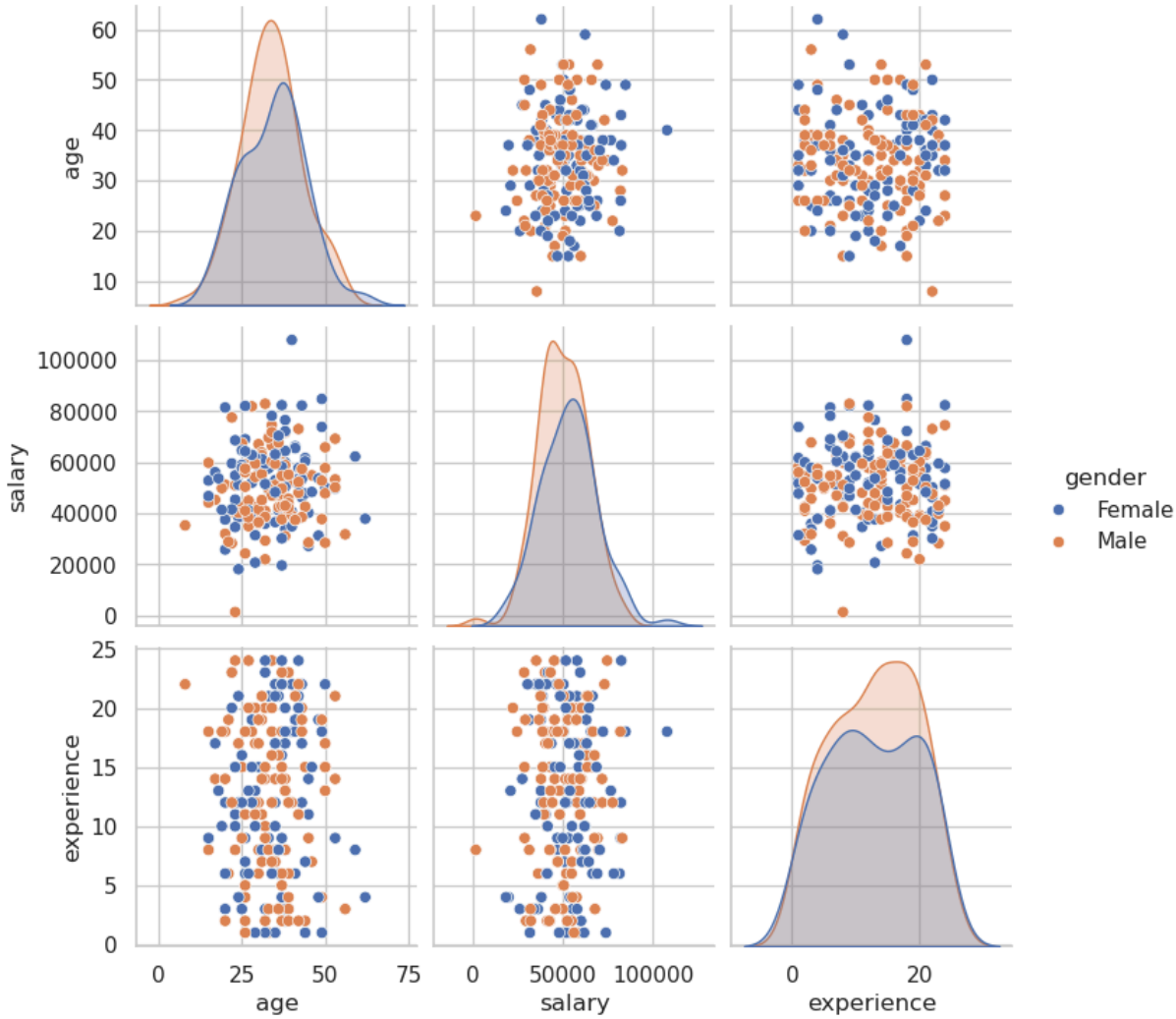
In [15]:
```
# Regression Plot
# sns.regplot(data=df, x="age", y="salary");
sns.lmplot(data=df, x='age', y='salary', hue='gender');
```



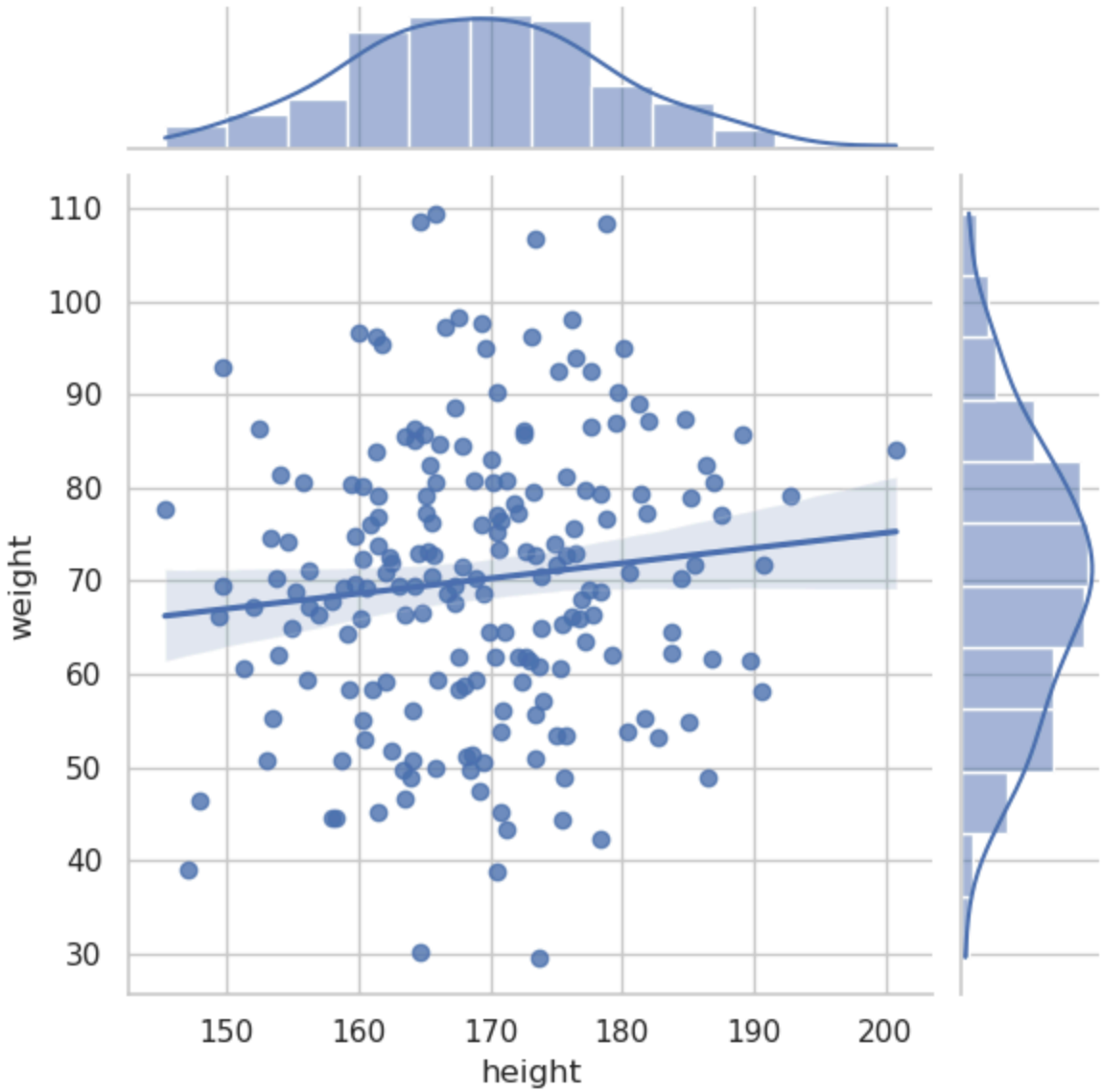
In [18]:
```
# Line Plot
# sns.lineplot(data=df, x='age', y='salary');
sns.lineplot(data=df, x='age', y='salary', hue='gender');
```



In [21]:
```
# Pair Plot (Pairwise relationships)
# sns.pairplot(df[['age', 'salary', 'experience']]);
sns.pairplot(df[['age', 'salary', 'experience', 'gender']], hue="gender");
```



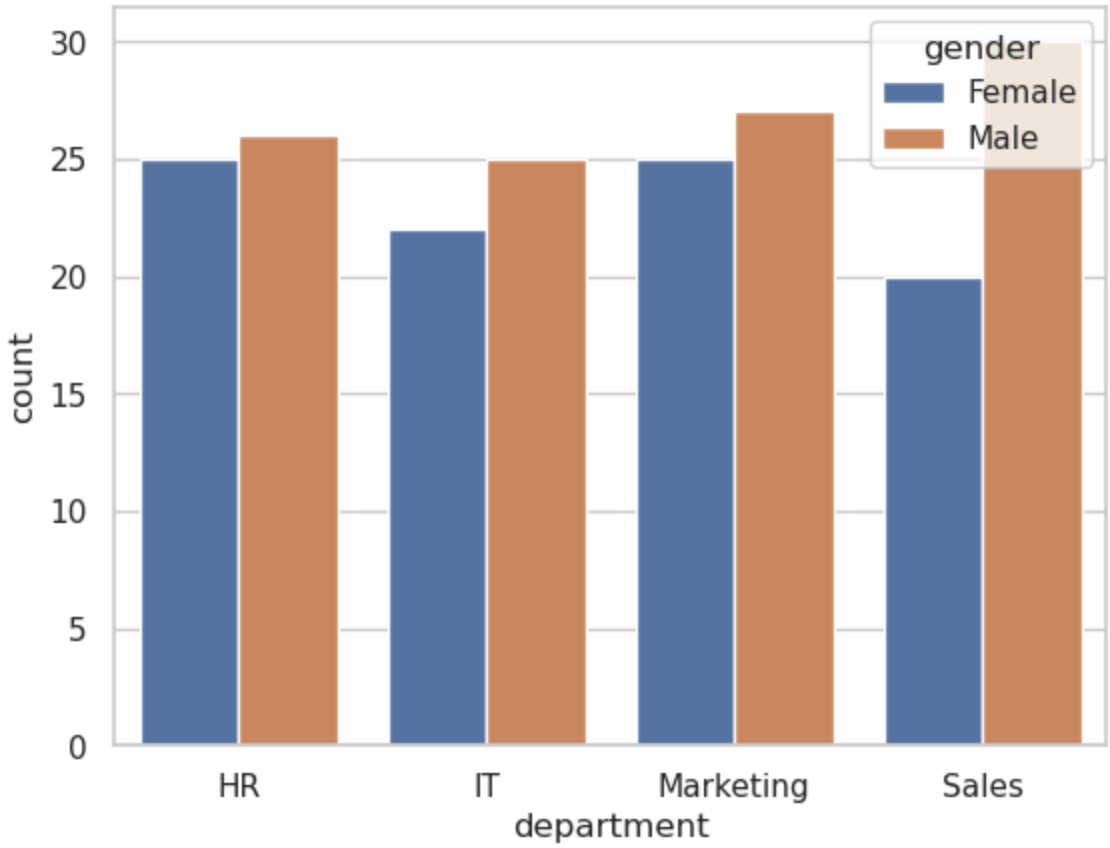
In [29]:
```
# Joint Plot (Scatter + Distribution)
# sns.jointplot(data=df, x="height", y="weight");
sns.jointplot(data=df, x="height", y="weight", kind='reg');
sns.jointplot(data=df, x="height", y="weight", hue='gender');
```



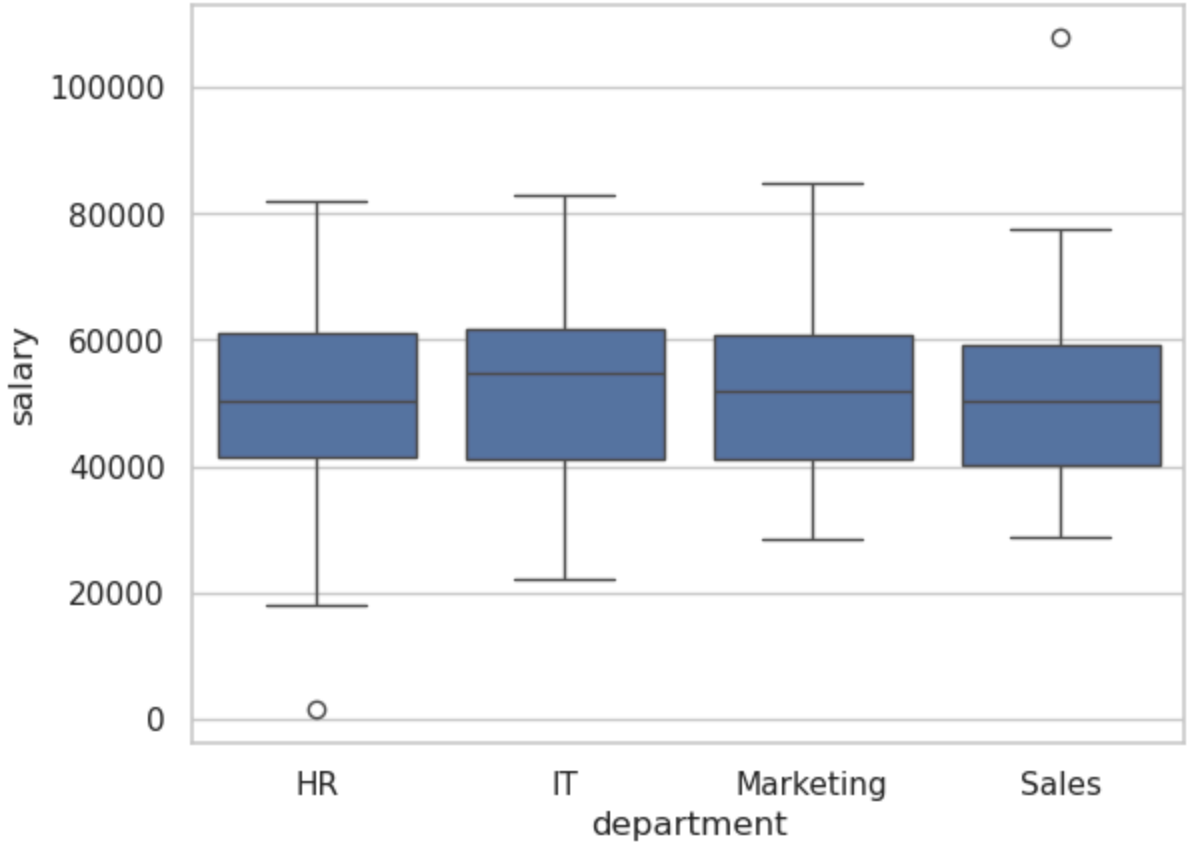
4. Categorical Plots

Used to compare a numeric variable across different categories.

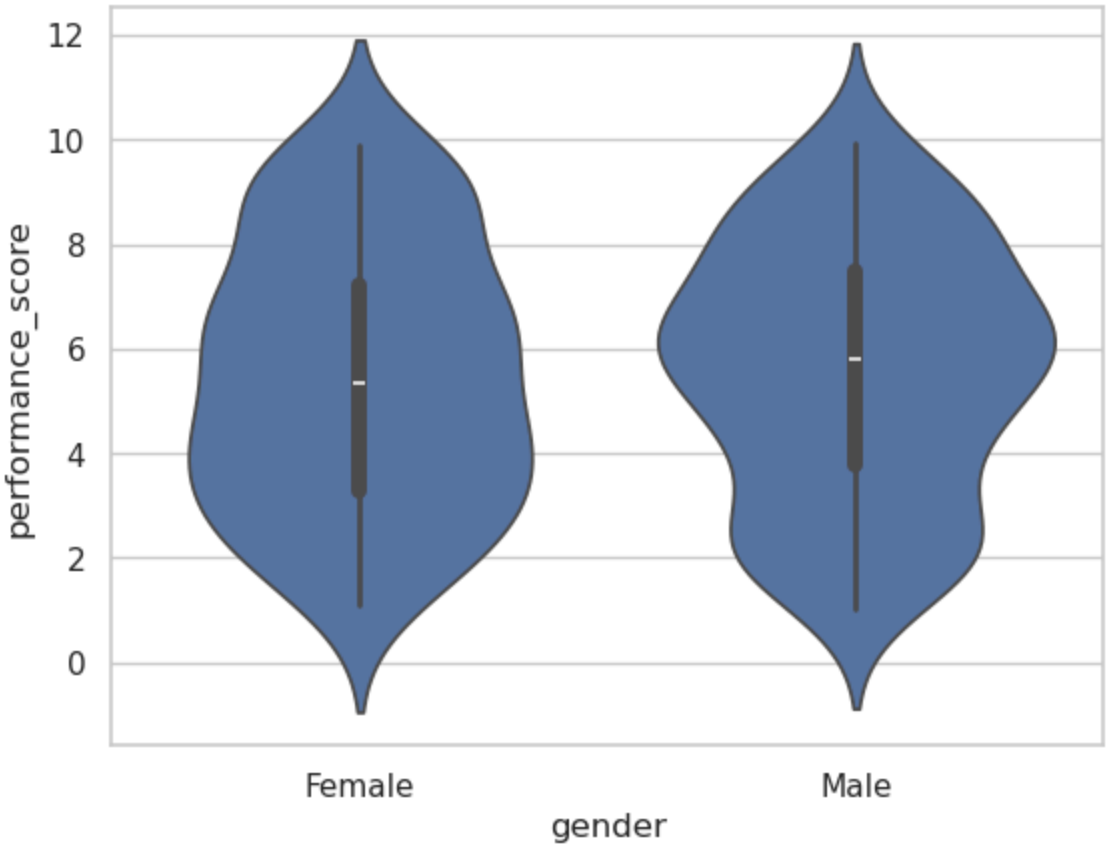
In [31]:
```
# Count Plot
# sns.countplot(data=df, x="department");
sns.countplot(data=df, x="department", hue="gender");
```



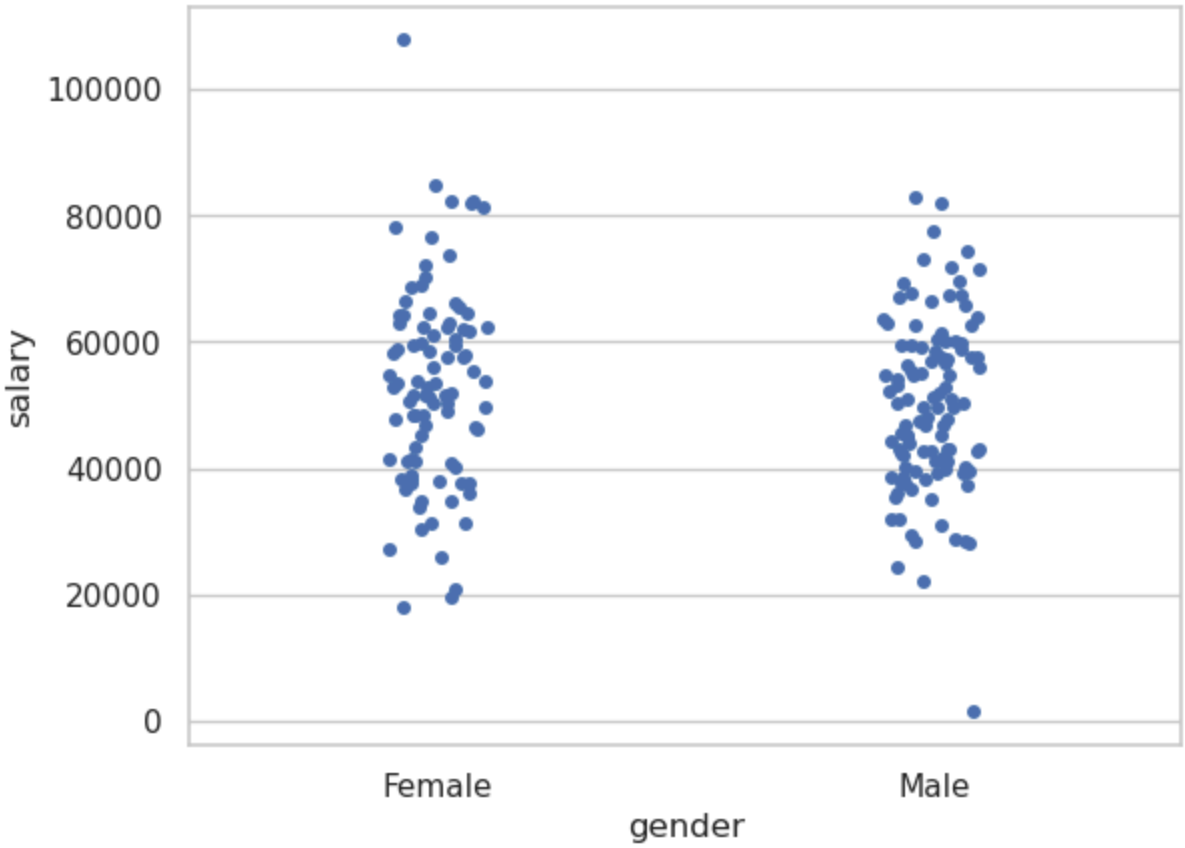
In [32]:
```
# Box Plot
sns.boxplot(data=df, x="department", y="salary");
```



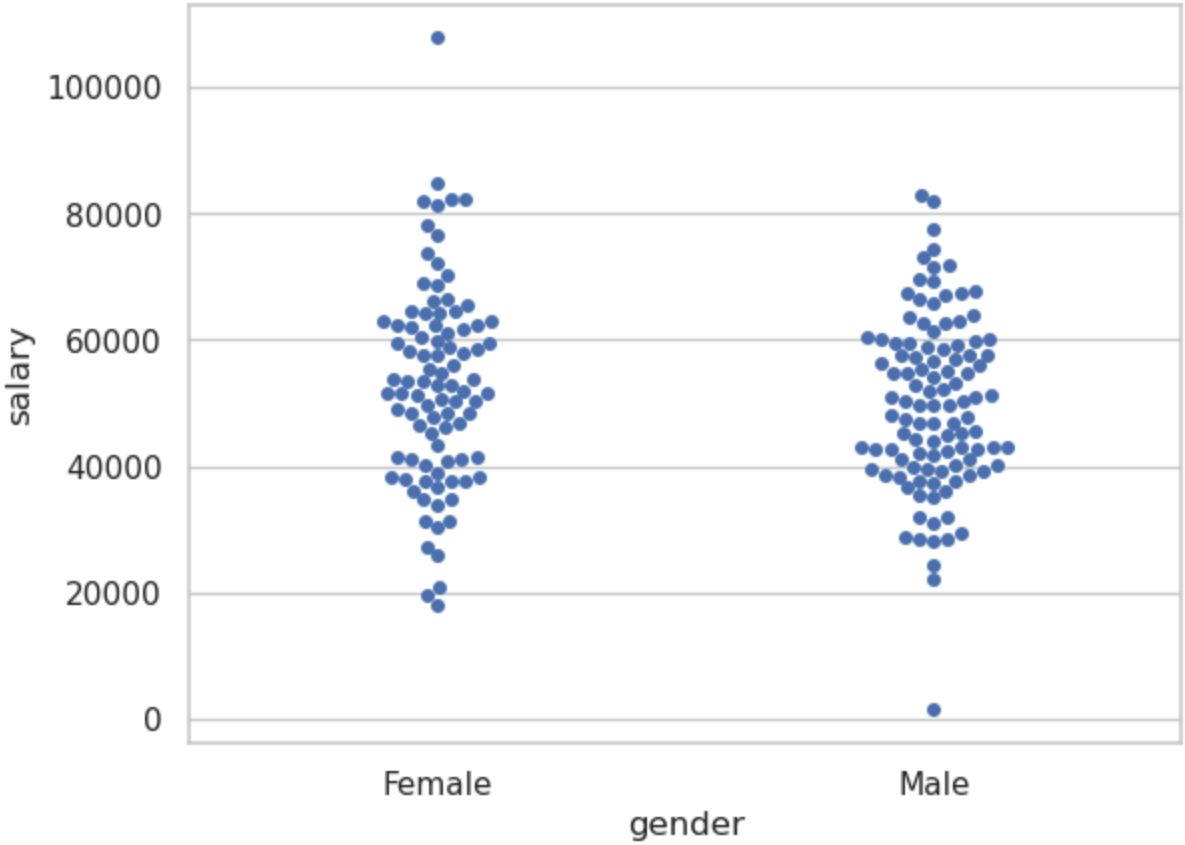
In [33]:
```
# Violin Plot
sns.violinplot(data=df, x="gender", y="performance_score");
```



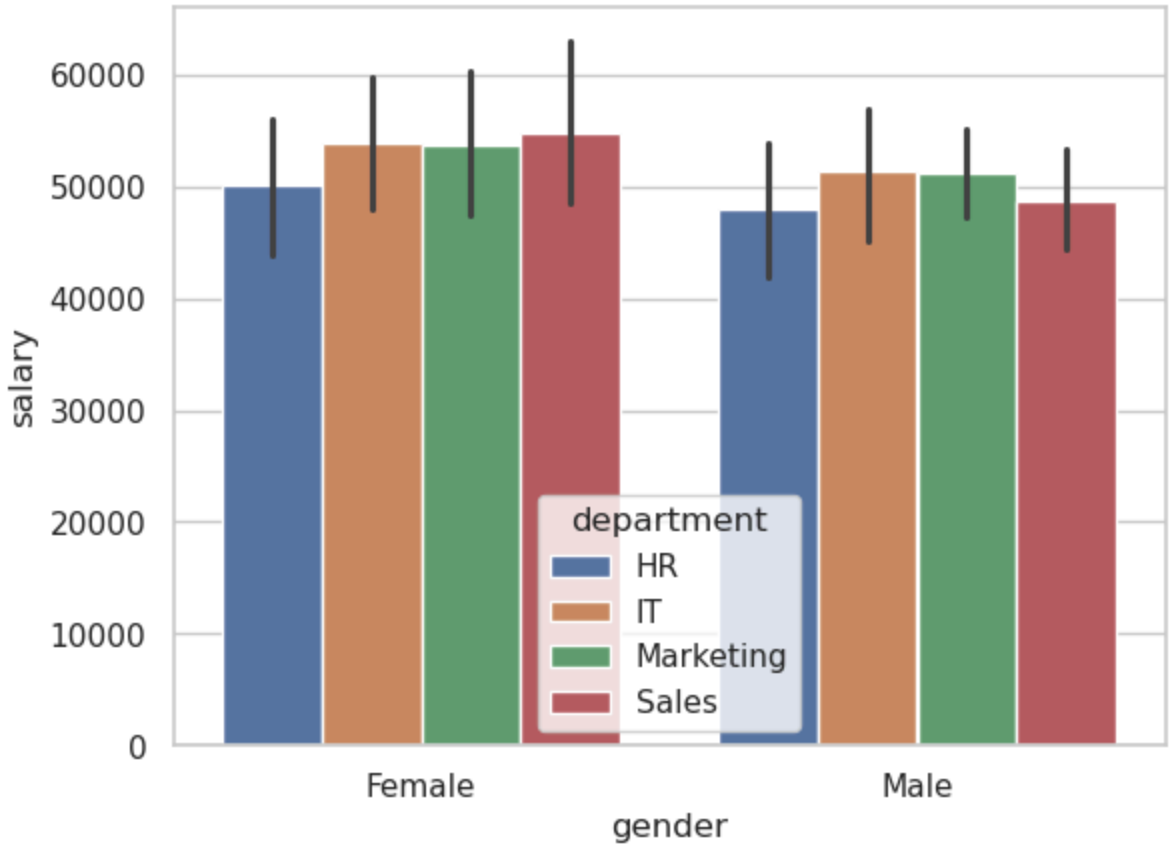
In [34]:
```
# Strip plot
sns.stripplot(data=df, x='gender', y='salary');
```



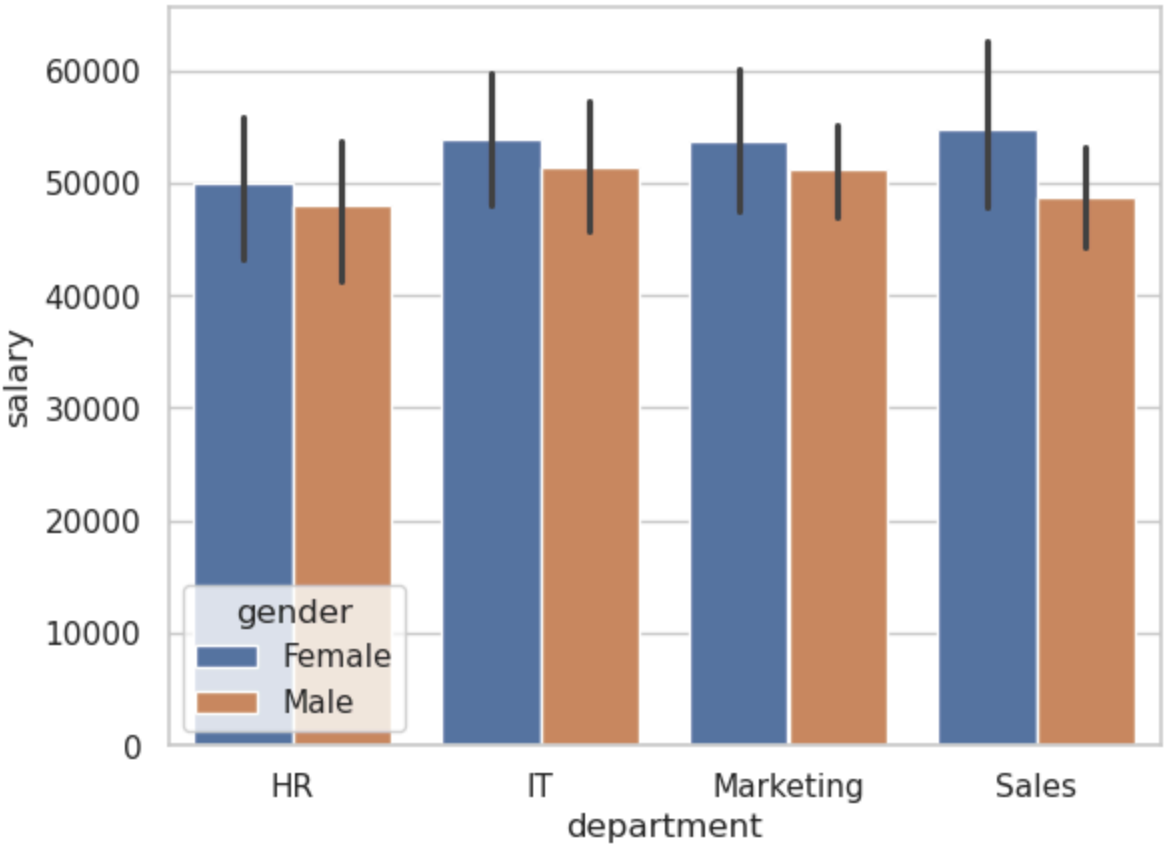
In [39]:
```
# Swarm Plot
sns.swarmplot(data=df, x="gender", y="salary");
```



In [44]:
```
# Bar plot
# sns.barplot(data=df, x="gender", y="salary");
sns.barplot(data=df, x="gender", y="salary", hue='department');
```



In [45]:
```
sns.barplot(data=df, x="department", y="salary", hue='gender');
```



5. Matrix Plots

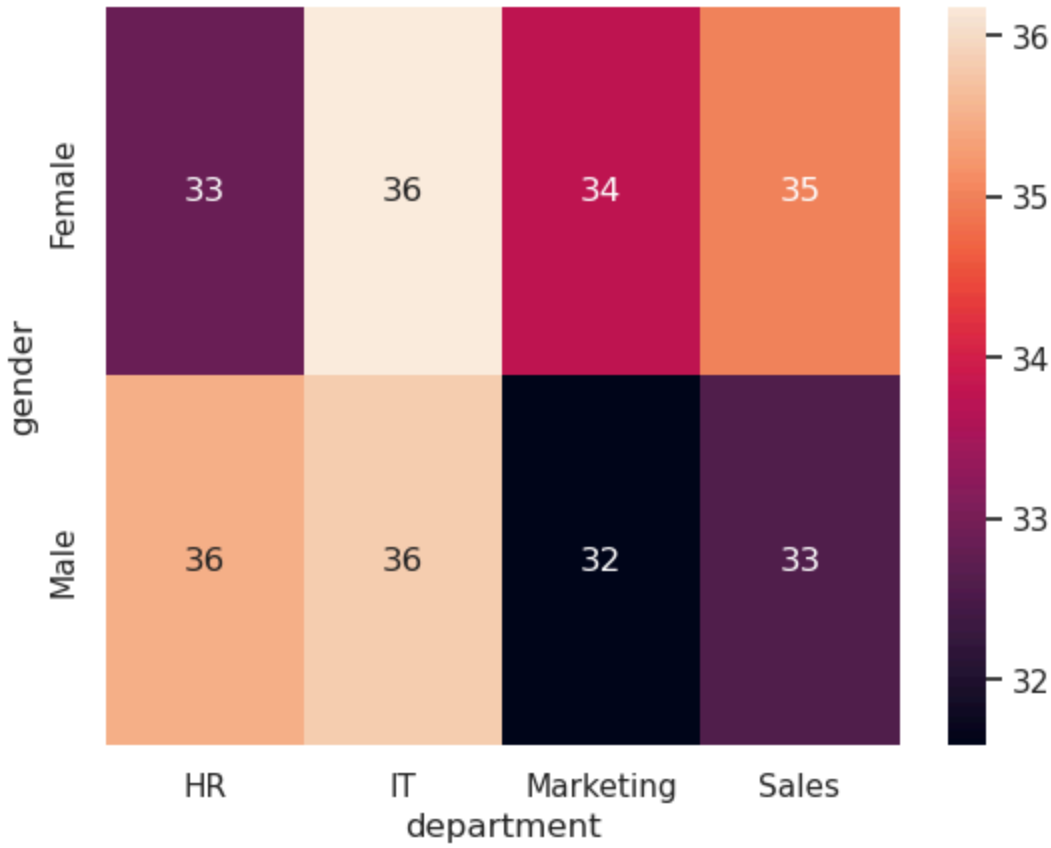
Ideal for showing correlation and hierarchical clustering.

In [54]:
```
tdf=df.pivot_table(index='gender', columns='department', values='age', obser
tdf
```

Out[54]:

department	HR	IT	Marketing	Sales
gender				
Female	32.84	36.181818	33.760000	35.000000
Male	35.50	35.840000	31.592593	32.566667

In [55]:
```
sns.heatmap(tdf, annot=True);
```

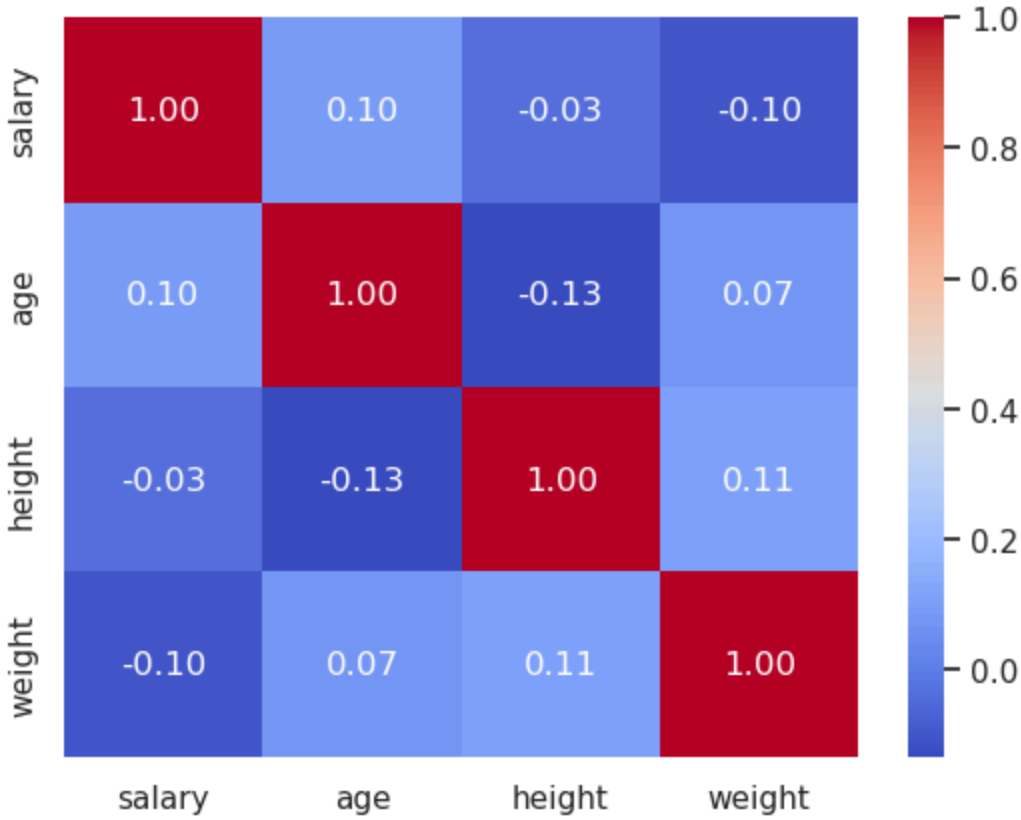


In [51]:
```
corr=df[['salary','age','height','weight']].corr()
corr
```

Out[51]:

	salary	age	height	weight
salary	1.000000	0.095547	-0.032626	-0.101396
age	0.095547	1.000000	-0.133682	0.068148
height	-0.032626	-0.133682	1.000000	0.105959
weight	-0.101396	0.068148	0.105959	1.000000

In [52]:
```
sns.heatmap(corr, annot=True, cmap="coolwarm", fmt=".2f");
```



In [ ]:
```
# Heatmap (Correlation)
corr = df.select_dtypes(include=[np.number]).corr()
# sns.heatmap(corr, annot=True, cmap="coolwarm", fmt=".2f");
```

6. Faceting (Small Multiples)

Breaking down plots into a grid based on categorical variables.

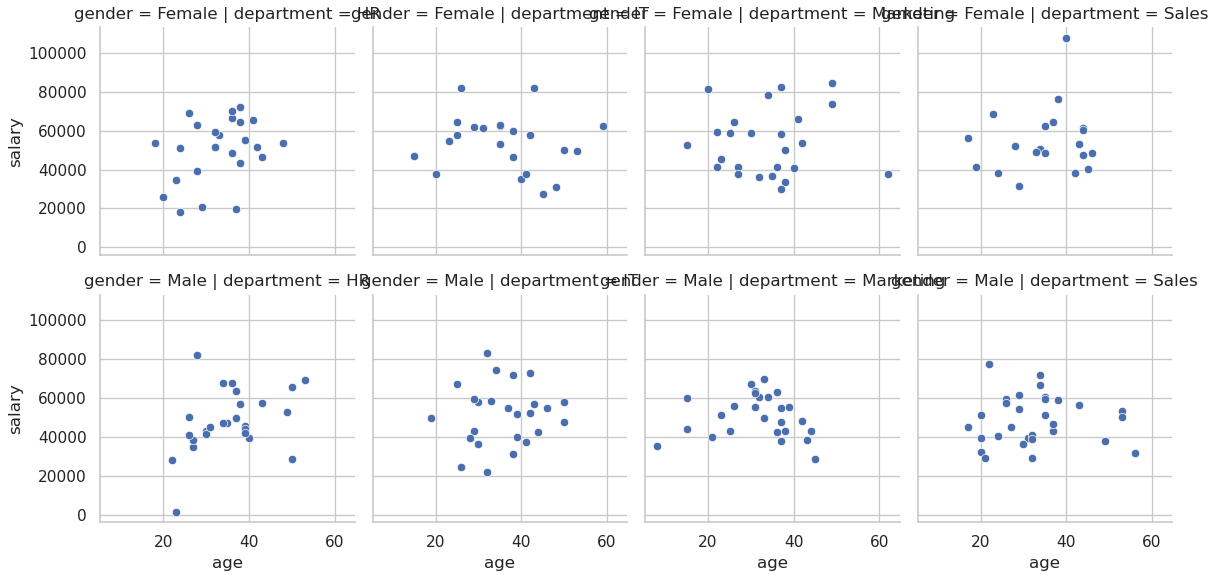
In [59]:
```
df.head()
```

Out[59]:

	age	salary	height	weight	gender	department	experience
0	39	55366.810405	154.055723	81.354829	Female	HR	3
1	33	58411.767896	164.006250	56.167520	Male	IT	1
2	41	66245.768648	170.052437	83.044089	Female	Marketing	6
3	50	65807.030781	170.469806	90.334568	Male	HR	15
4	32	29334.959481	165.499345	76.201524	Male	Sales	2

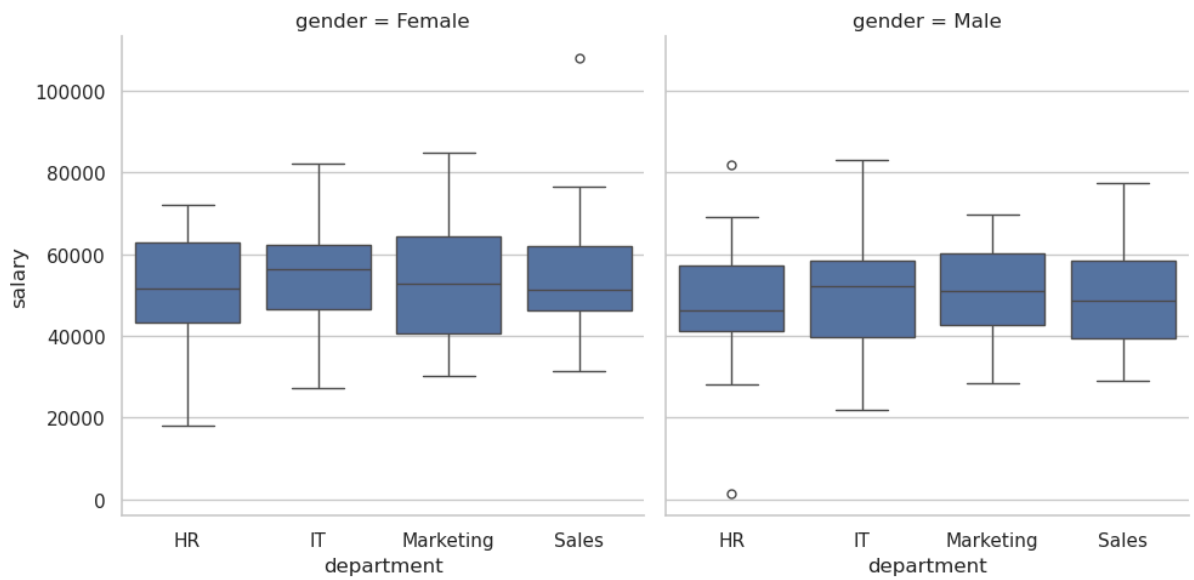
In [62]:
```
# FacetGrid Example
# sns.FacetGrid(df, row="gender", col="department").map(sns.histplot, "age")
# sns.FacetGrid(df, row="gender", col="department", hue='experience').map(sn
sns.FacetGrid(df, row="gender", col="department").map(sns.scatterplot, "age"
```

Out[62]:
```
<seaborn.axisgrid.FacetGrid at 0x7f3e39e08860>
```



In [67]:
```
# Categorical Wrapper (catplot)
# sns.catplot(data=df, x="department", y="salary");
sns.catplot(data=df, x="department", y="salary", col="gender", kind="box")
```

Out[67]:
```
<seaborn.axisgrid.FacetGrid at 0x7f3e38aa1b50>
```



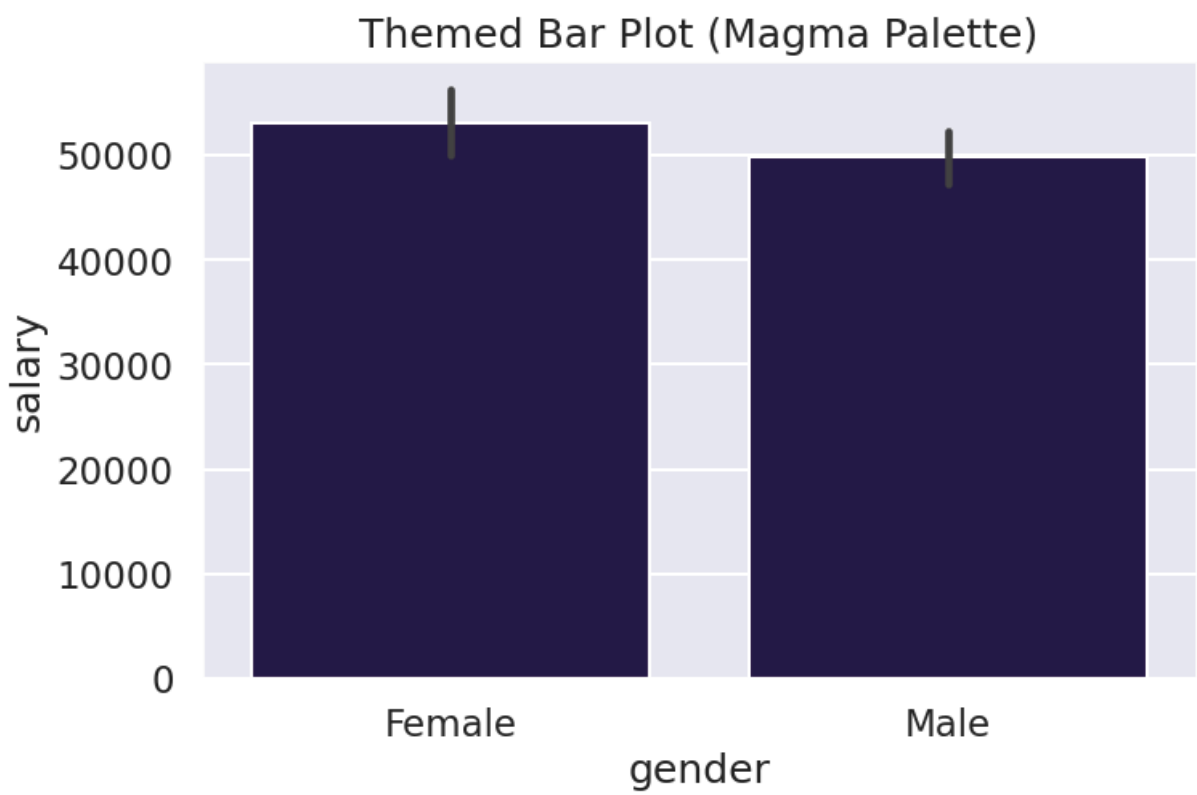
7. Theming and Customization

Changing the look and scale of your visualizations.

In [68]:
```
# Change context and style
sns.set_context("talk")  # Scales labels for presentations
sns.set_style("darkgrid")
sns.set_palette("magma")

plt.figure(figsize=(8,5))
sns.barplot(data=df, x="gender", y="salary")
```

```
plt.title("Themed Bar Plot (Magma Palette)")
plt.show()
```



In [ ]: